
Pruebas De Software

- **Falla (failure):** Ocurre cuando el software se comporta de forma inesperada.
- **Defecto (fault):** existe en el código, encontrarlo puede llevar a causar una falla.
- **Error (error,mistake):** acción incorrecta de la acción humana, esto puede incluir un defecto que conlleve a una falla.

¿Qué es un Test Exitoso?

- **Test exitoso = descubre defectos.**
- No consiste en demostrar que el software funciona, sino en encontrar problemas.
- Un buen resultado es identificar comportamientos inesperados
- Cada vez que se corrigen defectos detectados, se pueden introducir nuevos defectos.

Propósito del Testing

- Detectar defectos antes de la entrega.
- Validar que el software cumple requisitos y expectativas.
- Aumentar la calidad y confiabilidad.
- Prevenir futuras fallas y costos.

¿Qué NO es el Testing?

- Demostrar que no hay errores.
- Demostrar que el software funciona correctamente y cumple sus funciones.

El Testing es un proceso destructivo que trata de encontrar posibles defectos en el software.

Principios de Pruebas de Software

Principio	Descripcion
Definición del resultado esperado	Toda prueba debe incluir la salida o resultado esperado.
Independencia del programador	Un programador debería evitar probar su propio código.
Independencia de la unidad	Una unidad de programación no debería probar sus propios programas.
Inspección rigurosa	Los resultados de cada prueba deben ser acuciosamente revisados.
Casos con entradas inválidas	Los casos de prueba deben considerar entradas inválidas e inesperadas, además de las válidas.
Verificación de lo indebido	Probar un programa no solo implica verificar que haga lo que debe, sino también que no haga lo que no debe.
Registro de pruebas	Evitar casos de prueba espontáneos que no dejen registro, ya que son una pérdida de tiempo.
Planificación realista	No planificar pruebas asumiendo que no se encontrarán defectos.
Concentración de defectos	Donde ya se detectaron defectos, es más probable encontrar aún más en esa misma sección.
Creatividad en testing	El testing es una tarea creativa y un desafío intelectual.

Aspectos Económicos del Testing

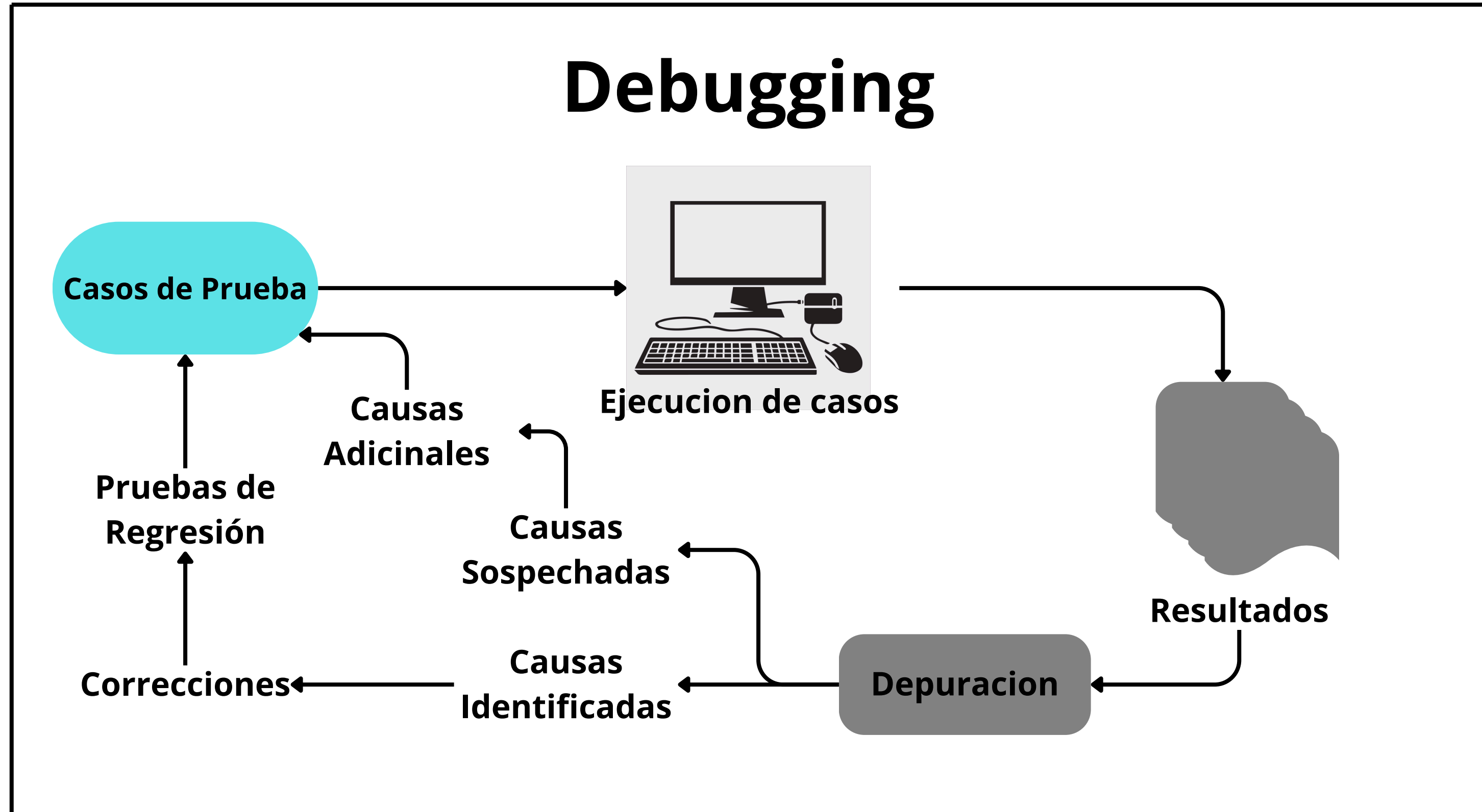
Es posible encontrar todos los defectos en un software/programa?

Aspectos Económicos del Testing

- Invertir en testing = ahorro significativo a largo plazo.
- Corregir defectos en fases tardías puede multiplicar los costos $\times 10$ o más.
- Testing bien planeado optimiza el retorno de inversión (ROI).
- Priorizar pruebas en módulos críticos maximiza beneficios

Testing vs. Debugging

	Testing	Debugging
¿Qué es?	Proceso de buscar defectos; validación	Proceso de analizar/identificar causa raíz
Objetivo	Encontrar que hay un problema	Descubrir y corregir el origen del problema
Quién participa	Tester, equipo de QA	Desarrollador
Cuándo	En diversas fases, de forma recurrente	Tras detectar defectos
Resultado	Reporte de defectos encontrados	Código corregido



Test Case

- Un buen Test Case es aquel que tiene un alto % de encontrar un defecto.
- No es demostrar que el software/programa funciona correctamente.
- No debe ser redundante, ni muy simple, ni complejo, solo debe ser el **mejor de su clase**.
- Diseñarlos es un gran desafío.
- **Incluye:** *datos de entrada, pasos a seguir, resultados esperados y criterios de aceptación.*

Diseño de Test Case

- Identificar requisitos y funcionalidades clave.
- Crear casos tanto positivos (flujos esperados) como negativos (errores, excepciones).
- Usar técnicas:
 - Caja negra.
 - Caja blanca.
- Revisar, mantener y actualizar los casos de prueba.

White-Box

- Se realizan a nivel de componentes.
- Normalmente guiadas por los requerimientos y las especificaciones técnicas.
- Son un procedimiento que verifica la lógica y la estructura interna del código de un programa, analizando:
 - Rutas.
 - Condiciones.
 - Bucles.
 - Cualquier control de flujo posible dentro del mismo.

White-Box

No es solo validar que el software funciona según se espera, sino también asegurar que cada camino/tramo/ruta del código este cubierto por al menos una prueba

White-Box

- Utiliza la estructura de control procedural, para derivar casos de prueba que:
 - Garanticen que todas las rutas de un modulo se ejercitaron por lo menos una vez.
 - Ejerciten todas las decisiones lógicas en sus lados True y False.
 - Ejecuten todos los loops en sus valores frontera y dentro de sus limites operacionales.

Objetivos

Garantizar que todas las rutas lógicas del código han sido ejecutadas y verificadas, asegurando así un mínimo de defectos a nivel estructural e implementacional.

- Cobertura Completa del Código.
- Detección de Errores Lógicos.
- Validación de Estructuras de Control.
- Optimización y Mejora del Código.
- Prevención de Vulnerabilidades.
- Automatización y Medición.

Razones para utilizarlo

- Detección Profunda de Defectos.
 - identifica errores que no son perceptibles desde una perspectiva externa.
- Garantía de Cobertura Alta.
 - Facilita métricas precisas sobre el grado de cobertura del código.
- Mejor Comprensión del Código.
 - El tester debe entender la implementación.
- Prevención de Código Inutilizado.
 - Ayuda a descubrir secciones de código no ejecutadas que pueden ser eliminadas
- Facilita la Refactorización.
 - revela oportunidades para mejorar algoritmos, lógica, etc.

Desventajas

- Alto Coste y Complejidad.
 - Se requiere comprender el código y su lógica, por ende se necesitan testers con perfiles especializados.
- Escalabilidad y Tiempo.
 - Es casi inviable cubrir todas las rutas en proyectos grandes.
- Mantenimiento de Pruebas.
 - Cambio de código → Actualización de pruebas.
- Ceguera Funcional
 - Riesgo de descuidar defectos de integración o de interacción con otros sistemas.
- Cobertura Incompleta de Requisitos.
 - cobertura estructural total $\neq$ Total Especificaciones Func. del sistema

Técnicas Utilizadas

- Cobertura de Sentencias
- Cobertura de Decisiones
- Cobertura de Ramas
- Cobertura de Caminos
- Cobertura de Condiciones y Condiciones Múltiples
- Cobertura de Bucles
- Pruebas de Flujo de Datos
- Pruebas de Mutación

Cobertura de Sentencias

Ejecutar todas las líneas (sentencias) del código al menos una vez.

```
if edad >= 18:  
    print("Es mayor de edad")  
else:  
    print("Es menor de edad")  
print("Fin del programa")
```

Cobertura de Decisiones

Ejecutar cada resultado posible (verdadero/falso) de todas las construcciones de decisión.

```
if (usuarioAutenticado) {  
    accesoSistema();  
}
```


Cobertura de Ramas

Ejecutar todos los posibles ramales de los bloques condicionales y de control de flujo.

```
if (x > 0) {  
    printf("Positivo\n");  
} else if (x < 0) {  
    printf("Negativo\n");  
} else {  
    printf("Cero\n");  
}
```

Cobertura de Caminos

```
if a > 0:
    if b > 0:
        print("A y B son positivos")
    else:
        print("A positivo, B no")
else:
    print("A no es positivo")
```

Cobertura de Condiciones

Evaluar todas las condiciones booleanas para los valores verdadero y falso.

```
if (A && B) {  
    // acción  
}
```

```
if (X > 0 && Y < 5) {  
    // Bloque A  
}
```


Cobertura de Bucles

Cubrir todas las combinaciones posibles de condiciones dentro de las decisiones compuestas.

```
for (int i = 0; i < n; i++) {  
    procesar(i);  
}
```

Pruebas de Flujo de Datos

Probar entradas que ejecuten bucles cero veces, una vez y múltiples veces.

```
if flag:
    x = 10
print(x)
```

Pruebas de Mutación

Introducir cambios ("mutantes") en el código para verificar si las pruebas existentes detectan el error simulado.

```
# Original  
if (a > b)  
  
#Mutado  
if (a < b)
```

Herramientas

Las herramientas sirven para automatizar y medir la cobertura de las pruebas de caja blanca.

Herramientas Comunes en python

- Hypothesis
- pytest-cov
- Pyre
- Coverage.py

Métricas de cobertura

- Porcentaje de sentencias cubiertas
- Porcentaje de ramas y caminos cubiertos
- Cobertura de condiciones y decisiones
- Cobertura de bucles

Black-Box

- Generalmente se dan a nivel de sistema y nivel de aceptación.
 - Pruebas funcionales.
 - Pruebas de regresión.
 - Pruebas de aceptación del usuario.
 - etc...
- No están limitadas a mandar datos y recibir resultados, sino también a la configuración de distintos ambientes que determinan la funcionalidad del software

Black-Box - Técnicas comunes

- Particionamiento de equivalencias
- Análisis de valores frontera
- Adivinanza de defectos
- Diagramas de causa-efecto
- Pruebas de transición de estado
- Matriz de grafos

Particionamiento de equivalencias

Consiste en dividir los datos de entrada en grupos o clases donde se espera que el sistema se comporte de manera similar para todos los valores dentro de cada clase. Se selecciona un valor representativo de cada clase para las pruebas.

Análisis de valores frontera

Complementa el particionamiento de equivalencias, pero se enfoca en probar específicamente los valores en los límites de cada clase, donde suelen ocurrir más errores.

Adivinanza de defectos

Técnica basada en la experiencia del probador para predecir dónde podrían ocurrir errores, utilizando conocimiento de errores comunes y problemas encontrados en el pasado.

Diagramas de causa-efecto

Técnica que representa gráficamente la relación entre las condiciones de entrada (causas) y los resultados esperados (efectos), utilizando operadores lógicos como AND, OR y NOT para determinar casos de prueba mínimos con máxima cobertura.

Pruebas de transición de estado

Técnica utilizada cuando el sistema puede estar en diferentes estados y las transiciones entre estos estados están gobernadas por reglas específicas. Se prueban las secuencias de eventos que provocan cambios de estado.

Matriz de grafos

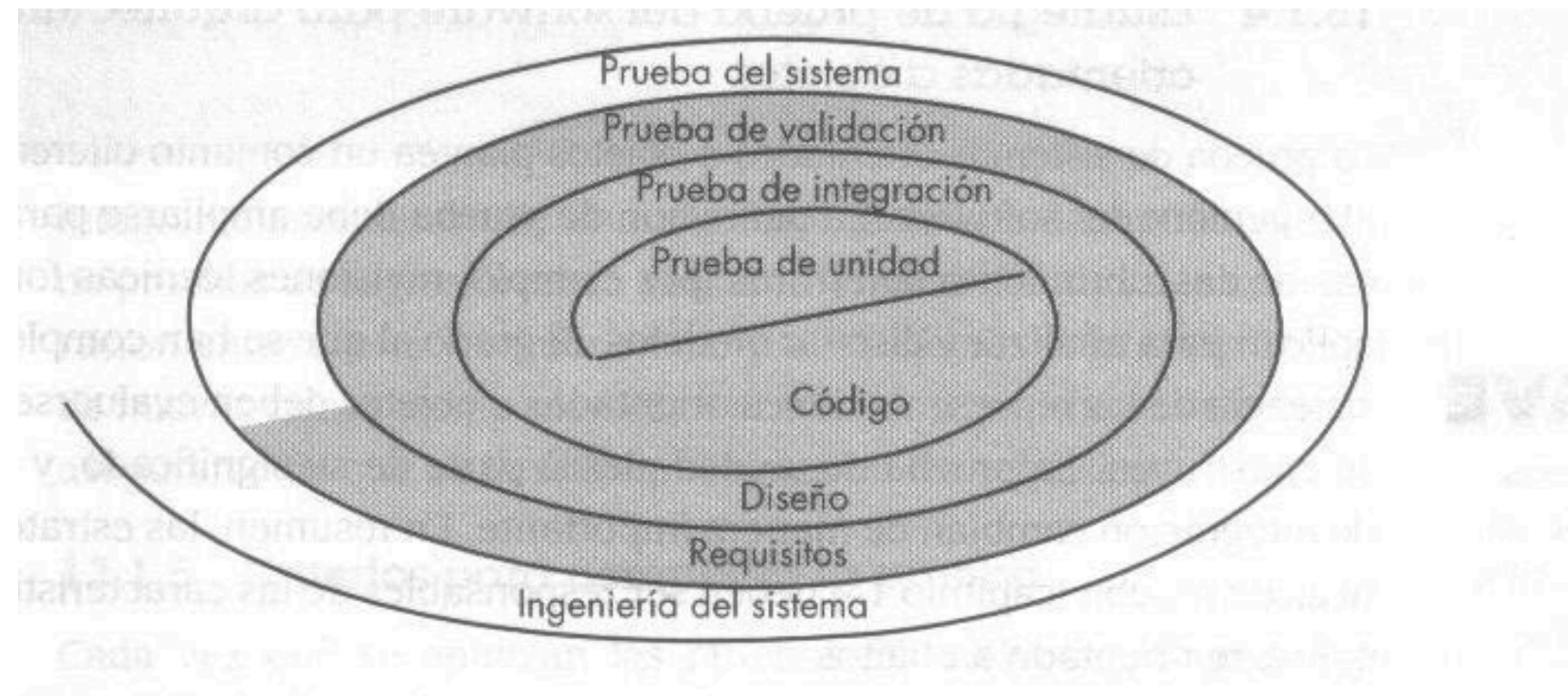
Representación tabular de las relaciones entre diferentes componentes o estados del sistema, utilizada para identificar y probar todas las posibles interacciones.

Diseño de Casos de Prueba

- No se debe descartar White-box por ser impráctico en grandes proyectos. Deben seleccionarse con cuidado los caminos lógicos importantes que se van a ejecutar.
- Se puede combinar Black-Box y White-Box para proveer un enfoque que valide la interfaz y a la vez asegurarse de que los aspectos internos son correctos.

Estrategias de Prueba

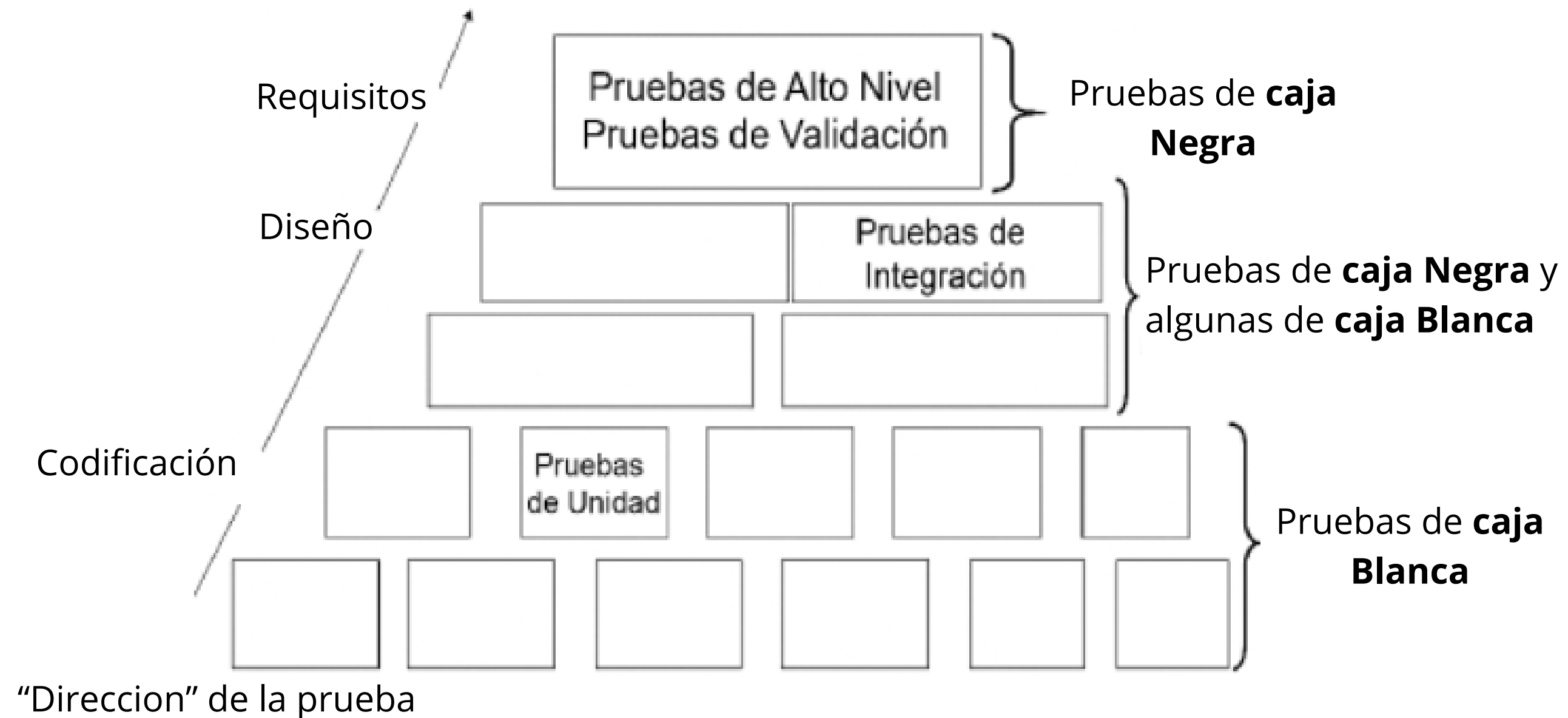
- Cuando creamos un software, comenzamos con la ingeniería del sistema, los requerimientos, etc.
 - Ósea que comenzamos desde afuera.
- Cuando realizamos las pruebas, tenemos que ir de dentro hacia afuera



Estrategias de Prueba

- Al inicio se deben ejecutar pruebas de White-Box de forma individual para cada modulo, asegurando que cada uno de estos funcione adecuadamente como una unidad.
- Luego, los módulos deben integrarse para así formar el software completo, aquí entran las técnicas de Black-Box.
- Para finalizar, corresponde realizar pruebas que aseguren que el software satisface los requerimientos de uso, funcionales, de comportamiento y de desempeño y verificar que todos los elementos del sistema se acoplan adecuadamente.

Estrategias de Prueba



Estrategias de Prueba

- Definir los requisitos del producto de manera medible antes de iniciar las pruebas.
- Dejar claros los objetivos de las pruebas.
- Conocer a los usuarios y crear perfiles de cada tipo.
- Hacer un plan que favorezca pruebas rápidas y continuas.
- Diseñar un software que pueda comprobarse a sí mismo.
- Realizar revisiones técnicas previas al testing.
- Revisar también la estrategia de pruebas y los casos definidos.
- Adoptar un enfoque de mejora continua en el proceso de pruebas.

Estrategias de Prueba

- Cobertura de requerimientos y funcionalidades debe realizarse con Black-Box Testing
- Cobertura Lógica debe emplear White-Box
- **Siempre deben realizarse pruebas que sean repetibles y definir resultados obtenidos**

Pruebas De Regresión

- Son procesos orientados a identificar si nuevos cambios en el código han introducido errores en funcionalidades que previamente eran estables.
- Se aplican tanto tras correcciones de bugs como después de incluir nuevas características.
- Apuntan a preservar la calidad y evitar que el software pierda funcionalidades ya validadas.

Estas pruebas deben diseñarse de tal manera de incluir solo pruebas de clases de errores de las funciones principales.

Se usan principalmente herramientas automatizadas de captura y ejecución

Objetivos

- Garantizar que **funcionalidades existentes no se vean afectadas por actualizaciones.**
- Minimizar la aparición de bugs recurrentes o “efectos colaterales”.
- Reducir riesgos en cada entrega, permitiendo **mejorar la velocidad y calidad del software.**

Tecnicas

Tecnica	Descripcion
Ejecucion Manual Seleccionada	Se ejecutan manualmente solo los casos afectados
Automotizacion Total	Se automatizan suites completas. Alta eficiencia y repetibilidad
Seleccion Basada En Riesgo	Solo se testean áreas críticas o afectadas
Priorizacion por impacto	Casos con mayor probabilidad/impacto de fallo van primero

Mejores prácticas

- Diseñar suites modulares, fáciles de actualizar y mantener.
- Priorizar la automatización de casos críticos y repetitivos.
- Integrar la ejecución de regresión en la pipeline de integración continua (CI/CD).
- Revisar y actualizar constantemente los casos de regresión.
- Evaluar métricas para identificar redundancias o coberturas insuficientes

Pruebas De Usabilidad

- Son evaluaciones realizadas para determinar cuán fácil, agradable e intuitivo es para los usuarios interactuar con el software.
- Se enfocan en la experiencia del usuario (UX), midiendo aspectos como facilidad de aprendizaje, eficiencia y satisfacción.
- Involucran la observación de usuarios reales realizando tareas típicas en la aplicación.

Objetivos

- Garantizar que los usuarios pueden completar sus objetivos principales de forma rápida e intuitiva.
- Identificar obstáculos, confusiones o elementos que dificultan la experiencia.
- Mejorar tasas de retención y percepción de valor.
- Detectar oportunidades de mejora en la interfaz y los flujos de interacción.

Métodos y métricas

Método de evaluación	Característica principal	Métrica asociada
Pruebas de usuario	Observación directa de tareas reales	Tasa de éxito, tiempo de tarea
Encuestas SUS	Cuestionario estándar de satisfacción (System Usability Scale)	Puntuación SUS
Análisis heurístico	Expertos detectan problemas mediante principios de usabilidad	Número y gravedad de problemas
Test A/B	Comparación de dos variantes reales	Conversiones, engagement, feedback

Pruebas De Validación o Aceptación

- Se enfocan en comprobar si el producto cumple con los requisitos y expectativas del cliente o usuario final.
- Constituyen el “filtro final”, validando que el software es apto para el entorno real antes de su adopción producción.
- Implican la participación directa o indirecta del cliente en la definición de criterios de aceptación y, muchas veces, en la ejecución.

Tipos principales de pruebas

- **Prueba de aceptación por el usuario (UAT):** Usuarios reales testean funcionalidades críticas en condiciones reales.
- **Prueba de aceptación del sistema:** Valida que el sistema cumple con los acuerdos contractuales y requisitos funcionales.
- **Prueba de aceptación operativa:** Se simulan escenarios de producción para asegurar operabilidad, rendimiento y soporte.

¿Cuándo comienzan?

- Se realizan tras finalizar las pruebas de verificación.
- Comienzan cuando el sistema está completamente integrado y funcional.
- Se enfocan en evaluar el producto desde la perspectiva del usuario final.
- Buscan responder:
 - **“¿Estamos construyendo el producto correcto?”**

Ej: Después de completar las pruebas unitarias e integración de un sistema de ventas, se ejecutan pruebas de validación para confirmar que las funciones de compra, pago y reportes cumplen los requisitos del cliente.

Pruebas Alfa

- Características:
 - Se ejecutan en el entorno del desarrollador.
 - Participan testers internos o usuarios seleccionados dentro de la empresa.
 - Permiten detectar errores antes del lanzamiento público.
 - Se simulan condiciones reales de uso.

Ej: Una empresa de videojuegos prueba internamente su nuevo título con empleados voluntarios para corregir errores antes de liberar la demo pública.

Pruebas Beta

- Características:
 - Se realizan en el entorno real del usuario.
 - Participan usuarios externos o finales.
 - Permiten recopilar opiniones reales sobre rendimiento, usabilidad y errores.
 - Sirven para ajustar la versión final del producto.

Ej: Una aplicación móvil lanza una versión beta en Google Play para que los usuarios reporten fallos y sugieran mejoras antes del lanzamiento oficial.

Aspectos pruebas Alfa y Beta

Aspecto	Pruebas Alfa	Pruebas Beta
Entorno	Interno (empresa)	Real (usuarios finales)
Participantes	Empleados o testers	Usuarios reales
Objetivo	Detectar errores antes del lanzamiento	Obtener retroalimentación real
Ejemplo	Juego probado por el equipo interno	App beta publicada en la tienda

Criterios de Aceptación

Criterio de aceptación	Característica principal
Cumplimiento funcional	El software realiza correctamente las tareas definidas
Usabilidad y accesibilidad	El sistema es apto para usuarios objetivo e inclusivo
Rendimiento bajo carga	El sistema soporta los volúmenes y tiempos requeridos
Seguridad y privacidad	Protección de la información y cumplimiento de normativas
Documentación y soporte	Manuales, ayuda y soporte completos

Pruebas De Sistemas

- Son pruebas que validan el comportamiento del software como un todo integrado, comprobando todos sus componentes, interfaces y flujos de trabajo.
- Permiten detectar errores que solo emergen cuando los distintos módulos interactúan bajo condiciones reales o simuladas.
- Incluyen aspectos funcionales, de integración, seguridad, rendimiento y compatibilidad.

Objetivos

- Garantizar que todos los módulos y componentes colaboran sin errores ni conflictos.
- Evaluar el desempeño bajo cargas reales o extremas.
- Comprobar la robustez, estabilidad y seguridad del sistema ante amenazas y fallos imprevistos.
- Validar integración con sistemas externos, bases de datos y APIs.

Tipos y enfoques

Enfoque	Descripción breve
Pruebas funcionales	Verifican que cada función cumple especificaciones
Pruebas de integración	Evalúan la interacción de módulos
Pruebas de rendimiento	Miden tiempo de respuesta y estabilidad bajo carga
Pruebas de seguridad	Analizan vulnerabilidades y protección de datos
Pruebas de compatibilidad	Prueban el sistema en diversos entornos (OS, navegador, móvil)

Mediciones de Pruebas

- Preguntas...
 - ¿Estamos encontrando los defectos más críticos?
 - ¿Cuánto falta por probar realmente?
 - ¿Qué tan confiables son nuestros resultados de prueba?
 - ¿El esfuerzo invertido en probar está dando resultado?
 - ¿Podemos mejorar sin medir?

Mediciones de Pruebas

- Permiten responder preguntas clave como:
 - ¿Qué tan grande es realmente el esfuerzo necesario para probar un producto?
 - ¿Qué tan efectivas fueron las actividades de verificación y validación?
 - ¿Cómo fue la calidad del software:
 - Durante las pruebas?
 - En producción?
 - En comparación con otros proyectos?
 - ¿Cuántos defectos había antes de probar y cuántos quedaron luego en operación?
 - ¿En qué momento es razonable detener el proceso de testing?

Medición y planificación

- Da una idea de la complejidad del programa, esto ayuda para planear una mejor prueba, a poder predecir cuantos errores existen y en que lugar pueden ocurrir.
- Analizar el número y tipo de defectos encontrados da una visión clara de:
 - La eficacia del proceso de verificación.
 - Las debilidades del desarrollo que deben corregirse.
- Medir defectos **no solo evalúa el producto**, sino también la **calidad del proceso** que lo generó.

Medición y planificación

- Las mediciones de pruebas permiten evaluar de forma cuantitativa el progreso, la efectividad y el alcance del proceso de testing.
- Ayudan a determinar cuándo detener las pruebas y a valorar la calidad del producto.

Qué aportan las mediciones

- Indican el nivel de calidad alcanzado por el software.
- Proveen criterios de preparación para la entrega (release readiness).
- Se correlacionan con la satisfacción del usuario final.
- Actúan como predictores de errores restantes.
- Al estar normalizadas, permiten comparar resultados entre proyectos.
- Ofrecen una visión de la eficiencia del proceso de prueba.

Mediciones Producen respuestas, y se debe priorizar tomando como base que es lo critico y que será utilizado

Tipos de mediciones útiles

- **Complejidad:** mide la dificultad o tamaño del software a probar.
- **Eficiencia de verificación:** evalúa la relación entre errores detectados y esfuerzo aplicado.
- **Cobertura de pruebas:** indica qué porcentaje del código o requisitos fue probado.
- **Ejecución de pruebas:** muestra el progreso y estado actual de las pruebas.
- **Defectos o incidentes:** cuantifica los errores encontrados y su tipo.
- **Métricas basadas en incidentes:** permiten analizar tendencias y mejorar procesos futuros.

Herramientas de Pruebas

- Facilitan el testing, y por tanto mas efectivo y productivo
 - Su objetivo es reducir esfuerzo manual, mejorar la precisión y aumentar la cobertura de las pruebas.
- Las herramientas de testing pueden clasificarse según la actividad en la que se usan, la función que cumplen o el área del proceso que apoyan.

Herramientas de Pruebas

Para revisiones e inspecciones

- **Objetivo:** detectar defectos tempranos en documentos, código o diseño antes de ejecutar pruebas.
- **Uso típico:** revisión de código, análisis estático, verificación de estándares o buenas prácticas.
- **Ejemplo:**
 - SonarQube → analiza el código fuente para encontrar errores, vulnerabilidades y problemas de estilo.
 - CodeClimate o Pylint también detectan defectos sin necesidad de ejecutar el programa.

Herramientas de Pruebas

Para la planificación de pruebas

- **Objetivo:** ayudar a definir estrategias, planes y cronogramas de pruebas, así como a organizar los recursos y casos de prueba.
- **Uso típico:** gestión de planes de prueba, control de avance, asignación de tareas y generación de reportes.
- **Ejemplo:**
 - TestLink → permite crear planes, casos de prueba y registrar resultados.
 - Zephyr o Xray (Jira) → integran la planificación de pruebas dentro de la gestión ágil del proyecto.

Herramientas de Pruebas

Para diseño y desarrollo de pruebas

- **Objetivo:** apoyar la creación y mantenimiento de casos de prueba, tanto manuales como automatizados.
- **Uso típico:** definir entradas, salidas esperadas y scripts automatizados.
- **Ejemplo:**
 - Selenium → automatiza pruebas funcionales de aplicaciones web.
 - JUnit (Java) o PyTest (Python) → permiten crear y ejecutar pruebas unitarias.

Herramientas de Pruebas

Para diseño ejecución de pruebas y evaluación de resultados

- **Objetivo:** automatizar la ejecución, recopilar resultados y evaluar el comportamiento del software bajo distintas condiciones.
- **Uso típico:** ejecución masiva de pruebas, simulación de usuarios, medición de rendimiento o carga.
- **Ejemplo:**
 - JMeter → mide el rendimiento y carga de aplicaciones.
 - Postman → ejecuta pruebas de API y valida respuestas automáticas.

Herramientas de Pruebas

Para soporte

- **Objetivo:** complementar el proceso de testing con funciones auxiliares, como gestión de defectos, comunicación o seguimiento.
- **Uso típico:** registrar y dar seguimiento a errores, priorizar fallos, generar reportes o gestionar versiones.
- **Ejemplo:**
 - Jira o Bugzilla → registran, asignan y hacen seguimiento a defectos.
 - Git o Jenkins → apoyan la integración continua y control de versiones.

Criterios de Termino

- Cuando se deja de realizar el testing?
 - Al agotarse los recursos.
 - En la practica nunca termina, solo cambia de tester.
 - Idealmente, utilizar modelos de fallas de software como función del tiempo de ejecución, basado en modelación estadística y teorías de confiabilidad para realizar afirmaciones como la sgte.:
 - *hemos hecho suficiente testing para afirmar con un 95% de confianza que la prob. de 1000 horas de GPU de operación libre de fallas en un ambiente probabilísticamente definido es por lo menos 0.995...*